S.B.M. Sr. Sec. School

School ID- 1516075

New Delhi-110015

Topic: - Complaint System 24x7

Session: 2024-2025

Guided By:-

Mr. Vikas Sharma

PGT (COMPUTER SCIENCE)

Submitted By:-

Arpit Patel

Class 12th B

CERTIFICATE

This is to certify that the computer science project titled '**Complaint System 24x7'** is done by **Arpit Patel** of Class 12th 'B' in the academic session of 2024-25, in the partial fulfilment of curriculum of C.B.S.E.'s examination 2024-25.

Teacher's Signature

_____

Principal's Signature

_____

_____

External Invigilator's Signature

ACKNOWLEDGEMENT

I would like to express special thanks of gratitude to our principal, **Dr. Preeti Shukla**, and my computer science teacher, **Mr. Vikas Sharma (PGT Computer Science)** to give me opportunity to work on the project of computer science on topic, **Complaint System 24x7**, which helped me to gain more knowledge and to build my practical thinking. It was very exciting and helpful.

I would also like to thank my family members and my friends, who helped me to work on this project.

I am also thankful to the entire computer science department of our school, for their support and cooperation. And, I am also thankful to our school Computer Science Lab for providing me enough resources to complete this project successfully.

DECLARATION

I hereby declare that the project work

Entitled:

"Complaint System 24x7"

Is prepared by:

Arpit Patel

CLASS 12th B

2024-25

Under the supervision of our Computer science teacher for the partial fulfillment of All India Senior Secondary Certificate Examination (AISSCE)

(2024-2025)

Table of Contents

OBJECTIVE

- The main objective to develop **Complaint System 24x7** is to simplify the task for handling the information of workers. This will provide an effective way to keep the data of worker and users safe.
- Our **Complaint System 24x7** deals with the various activities related to the worker and complaints. It allows the manager of any organization to add and find out the details of a worker and allocate them complaints according to their skillset, and allows the worker to keep their profile up to date.
- This **Complaint System 24x7** will be user-friendly and requires minimum human intervention. This helps the managers to register and access records of workers about their personal detail. It also manages the complaint history.

Project Category

- Python programming and MySQL database.

Resources Used

- Python IDLE
- MySQL 5.1
- Python and MySQL connector

About Frontend (Python)

Python is an object-oriented programming language that is designed in C. By nature, it is a high-level programming language that allows for the creation of both simple as well as complex operations.

HISTORY

Python was created in the early 1990s by Guido van Rossum at Stitching Mathematisch Centrum in the Netherlands as a successor of a language called ABC. Guido remains Python's principal author, although it includes many contributions from others.

In 1995, Guido van Rossum continued his work on Python at the Corporation for National Research Initiatives in Reston, Virginia where he released several versions of the software.

In May 2000, Guido van Rossum and the Python core development team moved to BeOpen.com to form the Be Open Python Labs team. In October of the same year, the Python Labs team moved to Digital Creations, which became Zope Corporation. In 2001, the Python Software Foundation was formed, a non-profit organization created specifically to own Python-related Intellectual Property. Zope Corporation was a sponsoring member of the PSF.

STORY BEHIND THE NAME:

Guido van Rossum, the creator of the Python language, named the language after the BBC show "Monty Python's Flying Circus". He doesn't particularly like snakes that kill animals for food by winding their long bodies around them and crushing them.

Features of Python

Simple and Easy: Python is a simple and minimalistic language. Reading a good Python program feels almost like reading English, although very strict English! Python is extremely easy to get started with. Python has an extraordinarily simple syntax.

Free and Open Source: Python is an example of Open Source Software. In simple terms, anyone can freely distribute copies of this software, read its source code, make changes to it, and use pieces of it in new free programs. OSS is based on the concept of a community which shares knowledge. This is one of the reasons why Python is so good - it has been created and is constantly improved by a community who just want to see a better Python.

Portable: Python programs work on any platforms without requiring any changes at all if you are careful enough to avoid any system-dependent features.

Interpreted: Python does not need compilation to binary. It just runs the program directly from the source code. Internally, Python converts the source code into an intermediate form called byte codes and then translates this into the native language of computer and then runs it.

Object Oriented: Python supports the procedure - oriented programming as well as object-oriented programming. In object-oriented languages, the program is built around objects which combine data and functionality.

Extensive Libraries: Along with this Python comes inbuilt with a wide array of modules as well as libraries which allows it to support many different programming languages like Java, C, C++, and JSON

About Backend (MySQL)

MySQL is a relational database management system (RDBMS) developed by Oracle that is based on structured query language (SQL).

MySQL is one of the most recognizable technologies in the modern big data ecosystem. Often called the most popular database and currently enjoying widespread, effective use regardless of industry, it's clear that anyone involved with enterprise data or general IT should at least aim for a basic familiarity of MySQL.

MySQL is integral to many of the most popular software stacks for building and maintaining everything from customer-facing web applications to powerful, data-driven B2B services. Its open-source nature, stability, and rich feature set, paired with ongoing development and support from Oracle, have meant that internet-critical organizations such as Facebook, Flickr, Twitter, Wikipedia, and YouTube all employ MySQL backend.

With **MySQL**, even those new to relational systems can immediately build fast, powerful and secure data storage systems. MySQL's programmatic syntax and interfaces are also perfect gateways into the wide world of other popular query languages and structured data stores.

Features of MySQL

Free and Open Source:

Any individual or enterprise may freely use, modify, publish, and expand on Oracle's open-source MySQL code base. The software is released under the GNU General Public License (GPL). For MySQL code needing to be integrated or included in a commercial application (or if open-source software is not apriority), enterprises can purchase a commercially licensed version from Oracle.

MySQL Is Easy To Use:

Though MySQL's relational nature and the ensuing rigid storage structures might seem restrictive, the tabular paradigm is perhaps the most intuitive, and ultimately allows for greater usability.

Compatible on Many Operating Systems:

MySQL is compatible to run on many operating systems, like Windows, Linux, etc.

Portable:

MySQL also provides a facility that the clients can run on the same computer as the server or on another computer.

High Flexibility:

MySQL supports a large number of embedded applications, which makes MySQL very flexible.

Memory Efficiency:

Its efficiency is high because it has a very low memory leakage problem.

The Future of MySQL

MySQL was originally envisioned to manage massive databases, faster than existing database software. Used in demanding operational, transactional, and production environments for decades, MySQL evolved alongside the move of computation and storage into the cloud.

Relative to many data storage and processing solutions on the market today, MySQL is an older technology, but it shows no signs of flagging in either popularity or utility. In fact, MySQL has enjoyed a recent resurgence over even more specialized modern storage systems, due to its speed, reliability, ease of use, and wide compatibility.

Hardware and System Requirements

- **OPERATING SYSTEM:**
  - WINDOWS 7 OR ABOVE
- **PROCESSOR:**
  - PENTIUM (ANY) OR AMD ATHALON (DUAL CORE)
- **RAM:**
  - 4 GB (RECOMMENDED)
- **HARD DISK:**
  - 6 GB (MINIMUM)
- **SOFTWARES AND LIBRARIES:**
  - **Front End:**
    - Python 3.X
    - Tabulate
  - **Back End:**
    - MySQL 5.X.X

Database Structure

```
MariaDB [(none)]> USE complaint_system_24x7;
Database changed
MariaDB [complaint_system_24x7]> SHOW TABLES;
+---------------------------------+
| Tables_in_complaint_system_24x7 |
+---------------------------------+
| complaint_history               |
| complaints                      |
| services                        |
| users                           |
+---------------------------------+
4 rows in set (0.000 sec)

MariaDB [complaint_system_24x7]> DESC complaint_history;
+--------------+-----------+------+-----+----------------------+----------------+
| Field        | Type      | Null | Key | Default              | Extra          |
+--------------+-----------+------+-----+----------------------+----------------+
| id           | int(11)   | NO   | PRI | NULL                 | auto_increment |
| user_id      | int(11)   | YES  | MUL | NULL                 |                |
| complaint_id | int(11)   | YES  | MUL | NULL                 |                |
| remarks      | text      | YES  |     | NULL                 |                |
| created_at   | timestamp | NO   |     | current_timestamp()  |                |
+--------------+-----------+------+-----+----------------------+----------------+
5 rows in set (0.009 sec)

MariaDB [complaint_system_24x7]> DESC services;
+-------------+--------------+------+-----+----------------------+----------------+
| Field       | Type         | Null | Key | Default              | Extra          |
+-------------+--------------+------+-----+----------------------+----------------+
| id          | int(11)      | NO   | PRI | NULL                 | auto_increment |
| name        | varchar(255) | NO   |     | NULL                 |                |
| description | text         | YES  |     | NULL                 |                |
| created_at  | timestamp    | NO   |     | current_timestamp()  |                |
+-------------+--------------+------+-----+----------------------+----------------+
4 rows in set (0.009 sec)

MariaDB [complaint_system_24x7]> DESC users;
+------------+-----------------------------------------+------+-----+----------------------+----------------+
| Field      | Type                                    | Null | Key | Default              | Extra          |
+------------+-----------------------------------------+------+-----+----------------------+----------------+
| user_id    | int(11)                                 | NO   | PRI | NULL                 | auto_increment |
| name       | varchar(255)                            | NO   |     | NULL                 |                |
| mobile     | varchar(15)                             | NO   | UNI | NULL                 |                |
| password   | varchar(255)                            | NO   |     | NULL                 |                |
| email      | varchar(255)                            | YES  |     | NULL                 |                |
| address    | text                                    | YES  |     | NULL                 |                |
| user_type  | enum('User','Worker','Manager','Admin') | YES  |     | User                 |                |
| manager_id | int(11)                                 | YES  | MUL | NULL                 |                |
| created_at | timestamp                               | NO   |     | current_timestamp()  |                |
+------------+-----------------------------------------+------+-----+----------------------+----------------+
9 rows in set (0.009 sec)
```

```
MariaDB [complaint_system_24x7]> DESC complaints;
+------------+---------------------------------+------+-----+---------------------+-------------------------------+
| Field      | Type                            | Null | Key | Default             | Extra                         |
+------------+---------------------------------+------+-----+---------------------+-------------------------------+
| id         | int(11)                         | NO   | PRI | NULL                | auto_increment                |
| user_id    | int(11)                         | YES  | MUL | NULL                |                               |
| worker_id  | int(11)                         | YES  | MUL | NULL                |                               |
| service_id | int(11)                         | YES  |     | NULL                |                               |
| details    | text                            | YES  |     | NULL                |                               |
| status     | enum('Pending','In Progress',   | YES  | MUL | Pending             |                               |
|            | 'Completed','Rated','Rejected') |      |     |                     |                               |
| created_at | timestamp                       | NO   |     | current_timestamp() |                               |
| updated_at | timestamp                       | NO   |     | current_timestamp() | on update current_timestamp() |
+------------+---------------------------------+------+-----+---------------------+-------------------------------+
8 rows in set (0.011 sec)
```

SOURCE CODE

```
import mysql.connector
from tabulate import tabulate


DB_HOST = 'localhost'
DB_NAME = 'complaint_system_24x7'
DB_USER = 'root'
DB_PASS = 'root'
# L_USER: Logged in User
conn = cursor = L_USER = None
def intro():
    print("\nWelcome To COMPLAINT SYSTEM 24x7\nMade By: Arpit Patel & Dilip\n")
    connect_db()
    main_menu()
def connect_db():
    global conn, cursor
    try:
        conn = mysql.connector.connect(host = DB_HOST, user = DB_USER, password
= DB_PASS)
        cursor = conn.cursor(dictionary = True)
        cursor.execute(f"SHOW DATABASES LIKE '{DB_NAME}';")
        if cursor.fetchone():
            cursor.execute(f"USE {DB_NAME};")
        else:
            create_db()
            insert_data()
    except mysql.connector.Error as err:
        closeApp(error = f"Database connection error: {err}")
    except Exception as e:
        closeApp(error = f"Error: {e}")
def create_db():
```

```
    cursor.execute(f"CREATE DATABASE {DB_NAME};")
    cursor.execute(f"USE {DB_NAME};")
    cursor.execute("""
        CREATE TABLE users (
            user_id INT PRIMARY KEY AUTO_INCREMENT,
            name VARCHAR(255) NOT NULL,
            mobile VARCHAR(15) NOT NULL UNIQUE,
            password VARCHAR(255) NOT NULL,
            email VARCHAR(255),
            address TEXT,
            user_type ENUM('User','Worker','Manager','Admin') DEFAULT 'User',
            manager_id INT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (manager_id) REFERENCES users(user_id)
        );
    """)
    cursor.execute("""
        CREATE TABLE complaints (
            id INT PRIMARY KEY AUTO_INCREMENT,
            user_id INT,
            worker_id INT DEFAULT NULL,
            service_id INT DEFAULT NULL,
            details TEXT,
            status ENUM('Pending','In Progress','Completed','Rated','Rejected') DEFAULT
'Pending',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES users(user_id),
            FOREIGN KEY (worker_id) REFERENCES users(user_id),
            INDEX (status)
        );
```

```
    """)
    cursor.execute("""
        CREATE TABLE complaint_history (
            id INT PRIMARY KEY AUTO_INCREMENT,
            user_id INT,
            complaint_id INT,
            remarks TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES users(user_id),
            FOREIGN KEY (complaint_id) REFERENCES complaints(id)
        );
    """)
    cursor.execute("""
        CREATE TABLE services (
            id INT PRIMARY KEY AUTO_INCREMENT,
            name VARCHAR(255) NOT NULL,
            description TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
    """)
    conn.commit()
def insert_table(table, data = {}):
    values = list(data.values())
    cursor.execute(f"INSERT INTO {table} (" + ",".join(list(data.keys())) + ") VALUES ( " + ",".join(["%s"] * len(values)) + ")", values)
    conn.commit()
    return cursor.lastrowid
def update_table(table, data = {}, where = {}):
    cursor.execute(f"UPDATE {table} SET " + ",".join([field + "=%s" for field in data.keys()]) + " WHERE " + " AND ".join([key + "=%s" for key in where.keys()]), list(data.values()) + list(where.values()))
    conn.commit()
```

```
    return cursor.rowcount

def insert_data():

    insert_table('users', { 'name': 'Admin', 'mobile': '1234567890', 'email':
"admin@complaint_system_24x7.com", 'password': "Password@123", 'address':
'Admin Address', 'user_type': 'Admin' })

    manager_id = insert_table('users', { 'name': 'Manager', 'mobile': '9876543210',
'password': "Password@123", 'user_type': 'Manager' })

    insert_table('users', {'name': 'Worker', 'mobile': '1111111111', 'password':
"Password@1111111111", 'user_type': 'Worker', 'manager_id': manager_id})

    user_id = insert_table('users', {'name': 'User', 'mobile': '2222222222', 'password':
"Password@2222222222", 'user_type': 'User'})



    for name, description in [

        ('Electrician', 'Electrician service.'),

        ('Plumber', 'Plumber service.'),

        ('Fan Repair', 'Fan repair service.'),

        ('Fan Installation', 'Fan installation service.'),

        ('Other', 'Other service.'),

    ]:

        service_id = insert_table('services', {'name': name, 'description': description})

        insert_table('complaints', {'user_id': user_id, 'service_id': service_id, 'details':
'Test Complaint for ' + name})



    conn.commit()

def main_menu():

    while True:

        try:

            L_USER and user_menu()

            valid_input('option', 'Choose an option:', options = {'Register': register,
'Login': login, 'Exit': closeApp}, title = 'Main Menu')()

        except KeyboardInterrupt:

            closeApp()

        except mysql.connector.Error as err:

            closeApp('MySQL error: ' + str(err))
```

```
        except Exception as err:
            closeApp('Error: ' + str(err))
def register():
    global L_USER
    print("\nEnter your details to register: ")
    user_id = add_update_user(success_msg = 'Registration successful!', error_msg =
'Registration failed. Please try again.')
    if user_id:
        return login(user_id)
    else:
        return False
def login(user_id = None):
    global L_USER
    if user_id:
        cursor.execute("SELECT * FROM users WHERE user_id=%s", (user_id,))
    else:
        mobile = valid_input('text', "Enter your mobile number", required = True)
        password = valid_input('text', 'Enter your password', required = True)
        cursor.execute("SELECT * FROM users WHERE mobile=%s AND password=%s",
(mobile, password))
    user = cursor.fetchone()
    if not user:
        print("\nLogin failed. Please try again.")
        return False
    del user['password']
    L_USER = user
    print("\nWelcome,", L_USER['name'], " !\n")
    return True
def refresh_db_connection():
    global conn, cursor
    try:
        cursor.close()
```

```
        conn.close()
        conn.connect()
        cursor = conn.cursor(dictionary = True)
        cursor.execute(f"USE {DB_NAME};")
    except mysql.connector.Error as err:
        closeApp(error = f"Database connection refresh error: {err}")

#### USER MENU ####
def user_menu():
    refresh_db_connection()
    if not L_USER:
        return False
    options = {
        'User': {
            'New Complaint': new_complaint,
            'Rate/Feedback Complaint': rate_complaint,
        },
        'Worker': {
            'Complete Complaint': complete_complaint,
            'Reject Complaint': reject_complaint,
        },
        'Manager': {
            'Assign Complaint': assign_complaint,
            'Complete Complaint': complete_complaint,
            'Reject Complaint': reject_complaint,
            'Add Worker': add_worker,
            'View Users/Workers': view_users_by_type,
        },
        'Admin': {
            'Add Manager': add_manager,
            'Add Service': add_service,
```

```
            'Update Service': update_service,
            'View Services': view_services,
            'View Users/Workers/Managers': view_users_by_type,
        },
    }[L_USER['user_type']]
    options.update({
        'View Complaints': view_complaints,
        'View Complaint History': view_complaint_history,
        'View Profile': view_profile,
        'Update Profile': update_profile,
        'Logout': logout,
    })
    valid_input('option', 'Choose an option:', options = options, title =
L_USER['user_type'] + ' Menu')()
    user_menu()


#### USER ####
def new_complaint():
    cursor.execute("SELECT * FROM services")
    list_data(cursor.fetchall(), 'No services found.', 'SERVICES')
    services = {service['id']: service['name'] for service in services}
    while True:
        service_id = valid_input('digits', 'Enter service ID', required = True)
        if service_id in services.keys():
            break
        print("\nInvalid service ID. Please try again.")
    complaint_id = insert_table('complaints', {
        'user_id': L_USER['user_id'],
        'service_id': service_id,
        'details': valid_input('text', 'Enter your complaint details', required = True),
    })
    print("\nComplaint submitted successfully!")
```

```
    add_complaint_history(complaint_id = complaint_id, remarks = 'Complaint submitted for ' + services[service_id])

def rate_complaint(complaint_id = None):
    complaint = get_complaint_details(complaint_id)
    if not complaint:
        return False
    if complaint['status'] in ['Rated', 'Rejected']:
        print(f"\nComplaint already {complaint['status']}.")
        return False
    if complaint['status'] not in ['Completed']:
        print("\nComplaint not completed yet.")
        return False
    while True:
        rating = valid_input('digits', "Enter rating (1-5)", required = True)
        if rating in range(1, 6):
            break
        print("\nInvalid rating. Please enter a number between 1 and 5.")
    update_complaint_status(complaint, 'Rated', f"Complaint rated: {rating}")


#### MANAGER ####
def assign_complaint():
    complaint = get_complaint_details()
    if not complaint:
        return False
    if complaint['worker_id']:
        print("\nComplaint already assigned to a Worker.")
        return False
    print("\nSelect a Worker to assign the complaint: ")
    view_users(user_type = 'Worker')
    worker_id = valid_input('digits', "Enter Worker ID", required = True)
    cursor.execute("SELECT * FROM users WHERE user_id=%s AND user_type='Worker' AND manager_id=%s",(worker_id, L_USER['user_id']))
```

```
    if not cursor.fetchone():
        print("\nWorker not found or not assigned to you. Please try again.")
        return False
    update_table('complaints', {'worker_id': worker_id}, {'id': complaint['id']})
    update_complaint_status(complaint, 'In Progress', 'Complaint assigned to Worker ID:' + worker_id)
def add_worker():
    user_id = add_update_user('Worker', manager_id = L_USER['user_id'])
    user_id and print("\nWorker Id: ", user_id)

#### WORKER/MANAGER ####
def complete_complaint():
    complaint = get_complaint_details()
    if not complaint:
        return False
    if not complaint['worker_id']:
        print("\nComplaint not assigned to any Worker.")
        return False
    if complaint['status'] not in ['Pending', 'In Progress']:
        print("\nComplaint already " + complaint['status'] + ". Cannot complete again.")
        return False
    update_complaint_status(complaint, 'Completed', 'Complaint completed')
def reject_complaint():
    complaint = get_complaint_details()
    if not complaint:
        return False
    if complaint['status'] not in ['Pending', 'In Progress']:
        print("\nComplaint already resolved.")
        return False
    update_complaint_status(complaint, 'Rejected', 'Complaint rejected')

#### ADMIN ####
```

```
def add_manager():
    user_id = add_update_user('Manager', manager_id = L_USER['user_id'])
    user_id and print("\nManager Id: ", user_id)
def add_service():
    service_id = insert_table('services', {
        'name': valid_input('text', 'Enter service name', required = True),
        'description': valid_input('text', 'Enter service description'),
    })
    print("\nService added successfully!\nService ID: ", service_id)
    view_services()
def view_services():
    cursor.execute("SELECT * FROM services")
    list_data(cursor.fetchall(), 'No services found.')
def update_service():
    view_services()
    service_id = valid_input('digits', "Enter service ID", required = True)
    cursor.execute("SELECT * FROM services WHERE id=%s",(service_id,))
    service = cursor.fetchone()
    if not service:
        print("\nService not found.")
        return False
    service_name = valid_input('text', "Enter new service name", default =
service['name'])
    service_description = valid_input('text', "Enter new service description", default =
service['description'])
    update_table('services', {'name': service_name, 'description':
service_description}, {'id': service_id})
    print("\nService updated successfully!\nService ID: ", service_id)
    view_services()


#### ADMIN/MANAGER ####
def view_users_by_type():
```

```
    option_names = ['All', 'User', 'Worker']
    if L_USER['user_type'] == 'Admin':
        option_names.append('Manager')
    view_users(valid_input('option', 'Select a user type to view', 'User',
        option_names = option_names,
        title = 'VIEW DETAILS',
    ))


#### COMMON ####
def add_update_user(user_type = 'User', user_id = None, manager_id = None,
success_msg = None, error_msg = None):
    try:
        if bool(user_id):
            cursor.execute('SELECT * FROM users WHERE user_id=%s', (user_id,))
            user = cursor.fetchone()
            value_error_if(not user, f"{user_type} not found.")
            field = valid_input('option', 'Select a field to update:', options = {
                'Name': 'name',
                'Mobile': 'mobile',
                'Password': 'password',
                'Email': 'email',
                'Address': 'address',
            }, required = True, title = 'UPDATE ' + user_type.upper())
            if field == 'password':
                value = valid_input(field, 'Enter new password') or user[field]
            else:
                value = valid_input(field, f"Enter new {field}", user[field])
            update_table('users', {field: value}, {'user_id': user_id})
            success_msg = success_msg or f"{user_type} updated successfully!"
        else:
            user_id = insert_table(table = 'users', data = {
                'name': valid_input('name', 'Enter name', required = True),
```

```
            'mobile': valid_input('mobile', 'Enter mobile number', required = True),
            'password': valid_input('password', 'Enter password', required = True),
            'email': valid_input('email', 'Enter email (optional)'),
            'address': valid_input('address', 'Enter address (optional)'),
            'user_type': user_type,
            'manager_id': manager_id,
        })
        success_msg = success_msg or f"{user_type} added successfully!"
    conn.commit()
    print("\n", success_msg)
    return user_id
  except Exception as e:
    error_msg = error_msg or 'Operation failed. Please try again.'
    print("\n", error_msg, "\nError: ", e)
    return None
def view_complaints():
  searching = valid_input('text', 'Do you want to search for a specific complaint? (y/n)')
  where = "1 "
  data = ()
  if searching == 'y':
    field = valid_input('option', 'Search Using Field:', options = {
        'Complaint ID': 'id',
        'User ID': 'user_id',
        'Worker ID': 'worker_id',
        'Service ID': 'service_id',
        'Status': 'status',
        'Details': 'details',
    }, title = 'SEARCH COMPLAINTS')
    if field == 'id':
        complaint = get_complaint_details()
        list_data(complaint, 'Complaint not found.', 'COMPLAINT DETAILS')
```

```
        print("\nComplaint History: ")
        return view_complaint_history(complaint['id'])
    if field in ['user_id', 'worker_id', 'service_id']:
        search_value = valid_input('digits', f"Enter the {field} to search")
    else:
        search_value = valid_input('text', 'Enter the value to search')
    if field == 'details':
        where += f" AND c.details LIKE %s"
        search_value = f"%{search_value}%"
    else:
        where += f" AND c.{field} = %s"
    data = (search_value,)
if L_USER['user_type'] in ['User', 'Worker']:
    where += f" AND c.{L_USER['user_type'].lower()}_id = {L_USER['user_id']}"
cursor.execute(f"""
    SELECT
        c.id,c.details,c.status,
        u.name AS user_name,
        u.mobile AS user_mobile,
        s.name AS service_name,
        w.name AS worker_name,
        w.mobile AS worker_mobile,
        updated_at
    FROM complaints c
    LEFT JOIN users u ON c.user_id = u.user_id
    LEFT JOIN users w ON c.worker_id = w.user_id
    LEFT JOIN services s ON c.service_id = s.id
    WHERE {where}
    ORDER BY status ASC,updated_at DESC
""", data)
```

```
    complaints = cursor.fetchall()
    list_data(complaints, 'No complaints found.')
    if not complaints:
        L_USER['user_type'] == 'Worker' and print('Any complaints assigned to you will be shown here.')
        return False
def view_complaint_history(complaint_id = None):
    complaint = get_complaint_details(complaint_id)
    if not complaint:
        return False
    list_data({
        'id': complaint['id'],
        'details': complaint['details'],
        'status': complaint['status'],
        'created_at': complaint['created_at'],
        'updated_at': complaint['updated_at'],
    }, title = "Complaint Details: ")
    cursor.execute("""
        SELECT ch.id,ch.user_id,u.name AS name,u.mobile AS mobile,u.user_type,ch.remarks,ch.created_at
        FROM complaint_history ch,users u
        WHERE ch.complaint_id=%s AND ch.user_id = u.user_id
        ORDER BY ch.created_at DESC
    """,(complaint['id'],))
    list_data(cursor.fetchall(), 'No history found.')
def view_profile():
    list_data(L_USER, title = 'Profile Details: ')
    if L_USER['manager_id']:
        cursor.execute('SELECT name,mobile,email FROM users WHERE user_id =' + L_USER['manager_id'])
        list_data(cursor.fetchone(), 'Manager not found.', 'Manager Details: ')
def update_profile():
```

```
    global L_USER
    view_profile()
    add_update_user(user_id = L_USER['user_id'], success_msg = 'Profile updated successfully!', error_msg = 'Profile update failed. Please try again.')
    refresh_db_connection()
    login(L_USER['user_id'])
def logout():
    global L_USER
    L_USER = None
    print("\nLogged out successfully!")


#### FUNCTIONS ####
def view_users(user_type = 'User'):
    if user_type == 'All':
        user_type = None
    where = '1 '
    if user_type in ['User', 'Worker', 'Manager']:
        where += f" AND user_type='{user_type}'"
    if user_type in ['Worker', 'Manager']:
        where += f" AND manager_id = {L_USER['user_id']}"


    columns = 'user_id AS id, name, mobile, email, address'
    if where.find('user_type') == -1:
        columns += ',user_type'
    cursor.execute('SELECT ' + columns + ' FROM users WHERE ' + where)
    list_data(cursor.fetchall(), 'No ' + user_type + 's found.')
def get_complaint_details(complaint_id = None):
    if complaint_id is None:
        complaint_id = valid_input('digits', 'Enter complaint ID', required = True)
    sql = "SELECT * FROM complaints WHERE id=%s"
    if L_USER['user_type'] in ['User', 'Worker']:
        sql += f" AND {L_USER['user_type'].lower()}_id = {L_USER['user_id']}"
```

```
    cursor.execute(sql, (complaint_id,))
    complaint = cursor.fetchone()
    not complaint and print("\n",{
        'User': 'Complaint not found or not submitted by you.',
        'Worker': 'Complaint not found or not assigned to you.',
        'Manager': 'Complaint not found.',
    }[L_USER['user_type']])
    return complaint or False
def update_complaint_status(complaint, new_status, remarks):
    update_table('complaints', {'status': new_status}, {'id': complaint['id']})
    add_complaint_history(complaint, remarks = remarks)
    if cursor.rowcount > 0:
        print(f"\nComplaint {new_status.lower()} successfully!")
    else:
        print(f"\nComplaint {new_status.lower()} failed. Please try again.")
def add_complaint_history(complaint = None, complaint_id = None, remarks =
None):
    complaint = complaint or get_complaint_details(complaint_id)
    if not complaint:
        return False
    insert_table('complaint_history', {
        'complaint_id': complaint['id'],
        'user_id': L_USER['user_id'],
        'remarks': valid_input('text', 'Enter remarks', required = remarks is None,
default = remarks),
    })
    return cursor.rowcount > 0
def list_data(data, nodata = 'No data found.', title = None):
    if type(data) not in [list, dict] or len(data) == 0:
        print("\n", nodata)
        return
    title and print(title)
```

```
    headers = 'keys'
    if type(data) == list and all([type(item) == dict for item in data]):
        pass
    elif type(data) == dict:
        data = data.items()
        headers = ['-', '-']
    else:
        print("\nInvalid data format.")
        return
    print(tabulate(data, headers, tablefmt = 'grid'))
def closeApp(error = None):
    try:
        error and print("\nError: ", error)
        if error or input("\nAre you sure you want to exit application? (Enter 'y' to exit): ") == 'y':
            pass  # Continue closing the app
        else:
            return
        cursor and cursor.close()
        conn and conn.close()
        print("\n\nThank you for using COMPLAINT SYSTEM 24x7. We are glad to help you.\n\n")
        exit()
    except KeyboardInterrupt:
        closeApp()


#### VALIDATIONS ####
def valid_input(type, text, default = None, required = False, options = {}, option_names=[], title = None):
    while True:
        try:
            if not text.endswith(':'):
```

```
            if default:
                text += f" [{default}]"
            text += ':'

        title and print(f"\n{title}\n")
        if type == 'option':
            return input_options(text, options, option_names, default, required)
        value = input_value = input(text + ' ')
        if not value:
            value_error_if(required, "This field is required.")
            return default
        types = {
            'mobile': validate_mobile,
            'email': validate_email,
            'name': validate_name,
            'password': validate_password,
            'digits': validate_digits,
        }
        if type in types.keys():
            return (types[type])(value)
        return input_value
    except ValueError as e:
        print("\nError: ", e, "\n")
    except Exception as e:
        print("\nError: ", e, "\n")


def input_options(text = 'Select an option: ', options = {}, option_names = [],
default = None, required = False):
    if not option_names and options:
        option_names = list(options.keys())
    i = 1
    for name in option_names:
```

```
        print(f"{i}. {name}")
        i += 1
    print()
    value = input(text + ' ')
    if value:
        value_error_if((not value.isdigit() or 1 < int(value) > len(option_names)) and not value in option_names, 'Invalid choice. Please try again.')
        option_name = option_names[int(value) - 1]
    elif default:
        option_name = default
    else:
        value_error_if(True, 'Option is required.')
    if options:
        return options[option_name]
    else:
        return option_name
def validate_mobile(value):
    for char in [' ', '-', '+91', '+']:
        value = value.replace(char, "")
    if value[:3] == '091' and len(value) == 13:
        value = value[3:]
    if value[:2] == '91' and len(value) == 12:
        value = value[2:]
    len(value) > 0 and validate_digits(value, 'Mobile number must contain only digits.')
    value_error_if(len(value) != 10, 'Mobile number must be 10 digits long.')

    cursor.execute("SELECT * FROM users WHERE mobile=%s", (value,))
    value_error_if(cursor.fetchone(), 'Mobile number already registered with us.')

    return value
def validate_email(value):
    username, *rest = value.strip().split('@')
```

```
    value_error_if(value.count('@') != 1, 'Email must contain @ symbol and only once.')

    value_error_if(len(username) == 0, 'Email must contain username.')

    domain = rest[0] or ''

    value_error_if(len(domain) == 0, 'Email must contain domain.')

    value_error_if(domain.count('.') == 0 or '.' in [domain[0], domain[-1]], 'Invalid domain in email.')

    return value

def validate_name(value):

    value_error_if(not value.replace(' ', '').isalpha(), 'Name must contain only alphabets.')

    value_error_if(len(value) < 3, 'Name must be at least 3 characters long.')

    return value

def validate_password(value):

    value_error_if(len(value) < 8, 'Password must be at least 8 characters long.')

    value_error_if(not any(char.isdigit() for char in value), 'Password must contain at least one digit.')

    value_error_if(not any(char.isupper() for char in value), 'Password must contain at least one uppercase letter.')

    value_error_if(not any(char.islower() for char in value), 'Password must contain at least one lowercase letter.')

    return value

def validate_digits(value, msg = 'It must contain only digits.'):

    value_error_if(not value.isdigit(), msg)

    return int(value)

def value_error_if(condition, error):

    if condition:

        raise ValueError(error)


#### MAIN PROGRAM ####

intro()
```

SCREENSHOTS (ADMIN)

Admin Details

Name	Admin
Mobile	1234567890
Email	admin@complaint_system_24x7.com
Password	Password@123
Address	Organisation Address

Admin Login

```
Welcome To COMPLAINT SYSTEM 24x7
Made By: Arpit Patel & Dilip


Main Menu

1. Register
2. Login
3. Exit

Choose an option: 2
Enter your mobile number: 1234567890
Enter your password: Password@123

Welcome, Admin  !
```

Admin Menu

```
1. Add Manager
2. Add Service
3. Update Service
4. View Services
5. View Users/Workers/Managers
6. View Complaints
7. View Complaint History
8. View Profile
9. Update Profile
10. Logout
9. Update Profile
10. Logout

Choose an option:
```

Add Manager

```
Choose an option: 1
Enter name : Manager A
Enter mobile number : 111111111

Error: Mobile number must be 10 digits long.

Enter mobile number : 1111111111
Enter password : password1111111111
Enter email (optional) :
Enter address (optional) :

 Manager added successfully!

Manager Id: 2
```

Add Service

```
Choose an option: 2
Enter service name: Carpenter
Enter service description: Carpenter Services

Service added successfully!
Service ID:  6
+------+------------------+---------------------------+---------------------+
|   id | name             | description               | created_at          |
+======+==================+===========================+=====================+
|    1 | Electrician      | Electrician service.      | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    2 | Plumber          | Plumber service.          | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    3 | Fan Repair       | Fan repair service.       | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    4 | Fan Installation | Fan installation service. | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    5 | Other            | Other service.            | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    6 | Carpenter        | Carpenter Services        | 2025-01-24 17:13:10 |
+------+------------------+---------------------------+---------------------+
```

Update Service

```
Enter service ID: 6
Enter new service name [Carpenter]:
Enter new service description [Carpenter Services]: Carpenter

Service updated successfully!
```

View Services

```
Choose an option: 4
+------+------------------+---------------------------+---------------------+
|   id | name             | description               | created_at          |
+======+==================+===========================+=====================+
|    1 | Electrician      | Electrician service.      | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    2 | Plumber          | Plumber service.          | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    3 | Fan Repair       | Fan repair service.       | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    4 | Fan Installation | Fan installation service. | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
|    5 | Other            | Other service.            | 2025-01-24 17:11:19 |
+------+------------------+---------------------------+---------------------+
```

View Users/Workers/Managers

```
Choose an option: 5

VIEW USERS/WORKERS/MANAGERS

1. All
2. User
3. Worker
4. Manager

Select a user type to view [User] : 1
```

id	name	mobile	email	address	user_type
1	Administrator	9876543210	admin@sbmills.org	Shivaji Marg, New Delhi	Admin
2	Manager A	1111111111			Manager
3	WorkerA	1000000001			Worker
4	WorkerB	1000000002			Worker
5	WorkerC	1000000003	worker.c@gmail.com	Worker C Address	Worker
6	Arpit Patel	9999999999	delhiarpitpatel@gmail.com		User

SCREENSHOTS (MANAGER)

Manager Login

```
WELCOME TO COMPLAINT SYSTEM 24x7
MADE BY: Arpit Patel & Dilip

MAIN MENU
1. Register
2. Login
3. Exit
Choose an option : 2
Enter your mobile number : 1111111111
Enter your password : password1111111111

Welcome, Manager A !
```

Manager Menu

```
1. Assign Complaint
2. Complete Complaint
3. Reject Complaint
4. Add Worker
5. View Users/Workers
6. View Complaints
7. View Complaint History
8. View Profile
9. Update Profile
10. Logout
```

Add Worker

```
Choose an option: 4
Enter name : WorkerA
Enter mobile number : 1000000001
Enter password : password1000000001
Enter email (optional) :
Enter address (optional) :

 Worker added successfully!

Worker Id: 3
```

SCREENSHOTS (WORKER)

Worker Login

```
WELCOME TO COMPLAINT SYSTEM 24x7
MADE BY: Arpit Patel & Dilip
MAIN MENU

1. Register
2. Login
3. Exit

Choose an option : 2
Enter your mobile number : 1000000001
Enter your password : password1000000001

Welcome, WorkerA !
```

Worker Menu

```
1. Complete Complaint
2. Reject Complaint
3. View Complaints
4. View Complaint History
5. View Profile
6. Update Profile
7. Logout
```

SCREENSHOTS (USER)

User Registration

```
WELCOME TO COMPLAINT SYSTEM 24x7
MADE BY: Arpit Patel & Dilip

MAIN MENU
1. Register
2. Login
3. Exit

Choose an option : 1
Enter your details to register:
Enter name : Arpit Patel
Enter mobile number : 9999999999
Enter password : 9999999999
Enter email (optional) :
Enter address (optional) :
 Registration successful!

Welcome, Arpit Patel !
```

User Login

```
Choose an option : 2
Enter your mobile number : 9999999999
Enter your password : 9999999999

Welcome, Arpit Patel !
```

User Menu

```
1. New Complaint
2. Rate/Feedback Complaint
3. View Complaints
4. View Complaint History
5. View Profile
6. Update Profile
7. Logout
```

Submission

```
Choose an option: 1
SERVICES
+------+-----------------+------------------------+--------------------+
|   id | name            | description            | created_at         |
+======+=================+========================+====================+
|    1 | Electrician     | Electrician service.   | 2025-01-24 17:11:19 |
+------+-----------------+------------------------+--------------------+
|    2 | Plumber         | Plumber service.       | 2025-01-24 17:11:19 |
+------+-----------------+------------------------+--------------------+
|    3 | Fan Repair      | Fan repair service.    | 2025-01-24 17:11:19 |
+------+-----------------+------------------------+--------------------+
|    4 | Fan Installation | Fan installation service. | 2025-01-24 17:11:19 |
+------+-----------------+------------------------+--------------------+
|    5 | Other           | Other service.         | 2025-01-24 17:11:19 |
+------+-----------------+------------------------+--------------------+
|    6 | Carpenter       | Carpenter              | 2025-01-24 17:13:10 |
+------+-----------------+------------------------+--------------------+
Enter service ID: 1
Enter your complaint details: Socket Burnt

Complaint submitted successfully!
Enter remarks [Complaint submitted for Electrician]:
```

Complaints List

Choose an option: 3

id	user_id	worker_id	details	status	history
1	5	2	The Internet is too slow.	Completed	2024-12-17 21:12:06 - Worker A (Worker): Visited the user premises. They need to upgrade their connection. 2024-12-17 21:08:08 - Manager (Manager): Complaint assigned to Worker ID: 2 2024-12-17 17:14:59 - Arpit Patel (User): Complaint submitted
2	5		Norton Antivirus is not working....	Rejected	2024-12-17 21:16:25 - Manager (Manager): Your license has already expired. 2024-12-17 17:25:59 - Arpit Patel (User): Complaint submitted

Complaint History

```
Choose an option: 4

Enter complaint ID: 1
+------+-----------+-------------+-------------+-----------------------------------------------------------------------+---------------------+
|   id |   user_id | user_name   | user_type   | remarks                                                               | created_at          |
|------+-----------+-------------+-------------+-----------------------------------------------------------------------+---------------------|
|    4 |         2 | Worker A    | Worker      | Visited the user premises. They need to upgrade their connection.     | 2024-12-17 21:12:06 |
|    3 |         1 | Manager     | Manager     | Complaint assigned to Worker ID: 2                                    | 2024-12-17 21:08:08 |
|    1 |         5 | Arpit Patel | User        | Complaint submitted                                                   | 2024-12-17 17:14:59 |
+------+-----------+-------------+-------------+-----------------------------------------------------------------------+---------------------+

Enter complaint ID: 2
+------+-----------+-------------+-------------+-----------------------------------+---------------------+
|   id |   user_id | user_name   | user_type   | remarks                           | created_at          |
|------+-----------+-------------+-------------+-----------------------------------+---------------------|
|    5 |         1 | Manager     | Manager     | Your license has already expired. | 2024-12-17 21:16:25 |
|    2 |         5 | Arpit Patel | User        | Complaint submitted               | 2024-12-17 17:25:59 |
+------+-----------+-------------+-------------+-----------------------------------+---------------------+
```

Rate Complaint

```
Choose an option: 2

Enter complaint ID: 1

Enter rating (1-5): 4

Enter remarks for rating: The Packages are misguiding.

Complaint history added successfully!

Complaint rated successfully!
```

Assign Complaint (To Worker)

```
Choose an option: 2

Enter complaint ID: 1

Select a Worker to assign the complaint:
+-------------+----------+------------+----------------------+----------------------+
|   Worker_id | name     |     mobile | email                | address              |
|-------------+----------+------------+----------------------+----------------------|
|           2 | Worker A | 1111111111 |                      |                      |
|           3 | Worker B | 2222222222 | workerb@example.com  |                      |
|           4 | Worker C | 3333333333 | workerc@example.com  | Address of Worker C  |
+-------------+----------+------------+----------------------+----------------------+

Enter Worker ID: 2

Complaint history added successfully!

Complaint assigned successfully!
```

Complete Complaint

```
Choose an option: 2

Enter complaint ID: 1

Enter remarks for completion (default: Complaint completed): Visited the user premises. They need to upgrade their connection.

Complaint history added successfully!

Complaint completed successfully!
```

Reject Complaint

```
Choose an option: 4

Enter complaint ID: 2

Enter remarks for rejection (default: Complaint rejected): Your license has already expired.

Complaint history added successfully!

Complaint rejected successfully!
```

Profile View & Update

```
Choose an option: 6
Profile Details:
+------------+---------------------+
| -          | -                   |
+============+=====================+
| user_id    | 6                   |
+------------+---------------------+
| name       | arpit               |
+------------+---------------------+
| mobile     | 1111111113          |
+------------+---------------------+
| email      |                     |
+------------+---------------------+
| address    |                     |
+------------+---------------------+
| user_type  | User                |
+------------+---------------------+
| manager_id |                     |
+------------+---------------------+
| created_at | 2025-01-24 17:24:24 |
+------------+---------------------+

UPDATE USER

1. Name
2. Mobile
3. Password
4. Email
5. Address

Select a field to update: 1
Enter new name [arpit]: Arpit Patel

 Profile updated successfully!
```

LIMITATIONS

- Email OTP Verification
- Cannot reopen same complaint.

FUTURE SCOPE

- Payment For Complaints
- Add Refund Options
- Worker Verification
- Worker Performance Report

CONCLUSION

As for the conclusion, the objectives for this project were achieved and functioned well as the desired target. The **Complaint System 24x7** database works systematically and will make ease the managers in order to manage all the workers data in the system. This system will give a better performance in arranging the complaints information without having to do it manually. This system will help manager to access the data of workers, users and complaints faster and easier.

REFERENCES

- **Google.com**
- **Python.org**
- **Tabulate:** pypi.org/project/tabulate
- **MySQL**
- **Text Book:** Computer Science with Python